

Sistema de Reconocimiento de Especies y Detección de Anomalías en Fauna Bentónica Profunda

Bitácora Técnica del Proyecto

Trabajo Final Integrador — Inteligencia Computacional (IC415)

Este documento reconstruye, en orden cronológico, el desarrollo completo del proyecto: qué se implementó en cada etapa, en qué se basó cada decisión técnica, qué resultados se obtuvieron, y — en los puntos donde el plan original cambió — por qué cambió. Su objetivo es servir como respaldo de la argumentación técnica del informe final y como guía de preparación para la defensa oral.

Dataset: FathomNet (MBARI) — Expedición Talud Continental IV, CONICET

Clases: Asteroidea · Octopoda · Porifera · Scyphozoa

1. Contexto y Objetivo del Proyecto

1.1 El problema real

El CONICET, en colaboración con el Schmidt Ocean Institute, llevó a cabo la expedición **Talud Continental IV** en el Cañón de Mar del Plata, registrando horas de video submarino mediante Vehículos Operados Remotamente (ROV) a cientos de metros de profundidad. La revisión de ese material es hoy un proceso completamente manual: biólogos especializados revisan frame a frame para identificar y documentar especies, un proceso lento, propenso a fatiga visual, y dependiente de un recurso escaso (expertos humanos). Esta es la motivación real del proyecto: automatizar ese cuello de botella entre la capacidad de recolección de datos (alta, gracias al ROV) y la capacidad de procesamiento (baja, limitada a revisión humana).

1.2 Objetivo del sistema

Se definió un sistema que toma fragmentos de video ya grabados (del orden de horas), aplica un muestreo de frames, clasifica cada muestra entre un conjunto de especies conocidas, y **marca explícitamente como “Desconocido”** cualquier caso que no se ajuste a esas clases — para no perder potenciales hallazgos de especies nuevas, que es precisamente el tipo de descubrimiento que motivó la expedición. El resultado final es un documento con las apariciones detectadas y su ubicación temporal en el video.

CONTEXTO CLAVE: Restricción de despliegue, no de algoritmo

El sistema corre en las **computadoras del buque**, no en el ROV, y procesa video ya grabado de forma **batch/offline** — no en tiempo real. Esta restricción aparentemente menor termina siendo determinante en decisiones posteriores (Fase 4): **la velocidad de inferencia, que suele ser el criterio dominante en visión por computadora aplicada, aquí pasa a un plano secundario.**

1.3 Datos y clases

Las imágenes se extrajeron de **FathomNet** (base de datos pública curada por MBARI), recortando cada observación a su bounding box. Se trabajó con 4 clases — **Asteroidea, Octopoda, Porifera y Scyphozoa** — con un dataset final de 1600 recortes (400 por clase antes de los descartes de integridad), dividido en una etapa posterior en train/val sin fuga de datos.

2. Fase 1-2: Pipeline de Datos y Clasificador Base (EfficientNet-B0)

2.1 Extracción y preparación de datos (Celdas 1-8)

Celda 1-3 — Entorno y extracción

Se montó Google Drive como almacenamiento persistente (todo el dataset, manifiestos y modelos viven ahí, no en el almacenamiento efímero de Colab). La extracción desde FathomNet (Celda 3) se construyó con **idempotencia robusta**: si una imagen ya fue descargada en una corrida anterior, no se vuelve a pedir — principio que más adelante se reutilizó para evitar reentrenamientos innecesarios (ver Sección 6).

Celda 4 — AutoEDA

El análisis exploratorio automático reveló dos hechos que condicionaron decisiones posteriores: (1) **desbalance entre clases**, resuelto luego con **WeightedRandomSampler** en el DataLoader; y (2) **completitud incompleta de metadatos oceanográficos** — profundidad (61.8%), temperatura (61.1%), tipo de imaging (72.1%) y plataforma (55.3%). Este segundo hallazgo resultó clave mucho más adelante (Sección 2.4).

Celda 5-6 — Integridad y split sin fuga de datos

Se descartaron imágenes corruptas (verificación de carga con Pillow). El split train/val se construyó con **StratifiedGroupKFold**, agrupando por UUID de imagen de origen: varios recortes pueden provenir de la misma foto (comparten fondo, iluminación, individuo), y un split puramente aleatorio podría mandar recortes casi-duplicados a ambos lados, inflando artificialmente las métricas de validación. Verificación dura aplicada: 0 imágenes compartidas entre train y val.

Celda 7-8 — Sanity check y DataLoaders

Inspección visual de lo que efectivamente “ve” la red tras el preprocesamiento (Letterboxing: redimensionado que preserva relación de aspecto sin recortar información, a diferencia de un resize directo que distorsiona la forma de los organismos). Los DataLoaders de PyTorch se construyeron sobre ImageFolder, con augmentation solo en el split de entrenamiento.

2.2 Transfer Learning y Fine-Tuning del Backbone

Celdas 9-10 — Entrenamiento de la cabeza

Se eligió **EfficientNet-B0** preentrenado en ImageNet como backbone inicial, por su buen balance precisión/eficiencia para hardware no especializado. Se congeló todo el backbone y se entrenó solo una cabeza clasificadora nueva de 4 clases durante 18 épocas, seleccionando el mejor checkpoint por **F1 macro de validación** (no por accuracy: con clases desbalanceadas, accuracy premia predecir siempre la clase mayoritaria).

POR QUÉ: F1 macro como criterio de selección de modelo

Con desbalance de clases, accuracy es una métrica engañosa: un modelo que siempre predice “Porifera” (la clase más numerosa) puede lograr accuracy alta sin haber aprendido nada de las clases minoritarias. F1 macro promedia el desempeño **por clase**, castigando explícitamente ese comportamiento degenerado.

Resultado: **F1 macro val = 0.863** tras la fase de cabeza únicamente.

Celda 12 — Fine-tuning completo con learning rates diferenciales

Se descongeló todo el backbone para un ajuste fino de 12 épocas con early stopping (**paciencia = 4**). Para mitigar *catastrophic forgetting* (el riesgo de que el backbone preentrenado “olvide” sus rasgos generales útiles

al recibir gradientes grandes), se usaron **learning rates diferenciales por bloque**: capas tempranas (rasgos genéricos: bordes, texturas) con LR ínfimo ($1e-6$), capas tardías con LR intermedio ($1e-5$), cabeza con LR mayor ($5e-5$).

Resultado: **F1 macro val = 0.910** (mejora de +0.047 sobre la fase de cabeza).

Celda 13 — Diagnóstico

Matriz de confusión (cruda y normalizada por fila) y F1 por clase, para identificar específicamente dónde el clasificador confunde categorías — insumo que se retomó más adelante al interpretar los patrones de falsa alarma del sistema de anomalías (Sección 3).

3. Fase 3: Reconocimiento de Conjunto Abierto (OSR)

La consigna del trabajo exige que el sistema etiquete como “Desconocido” cualquier anomalía — el clasificador de la Fase 2 solo sabe responder con una de las 4 clases conocidas, incluso ante una entrada completamente ajena a su entrenamiento (es un clasificador de *conjunto cerrado*). Esta fase construye la capa que falta: un detector de anomalías sobre el espacio de representaciones internas (*embeddings*) que la red ya aprendió.

3.1 Extracción de embeddings y visualización (Celdas 14-15)

Se “decapitó” el EfficientNet-B0 fine-tuneado (se reemplazó la cabeza clasificadora por una *capa identidad*), obteniendo el vector de **1280 dimensiones** que la red genera internamente para cada imagen, antes de clasificar. Sobre 1262 muestras de train y 338 de val se extrajeron estos vectores y se guardaron en Drive.

Para auditar visualmente la separación de clases se usó **UMAP** en lugar de t-SNE: UMAP preserva estructura local y global (las distancias entre clusters son interpretables), es determinista fijando semilla, y expone un método `.transform()` reutilizable para proyectar puntos nuevos — ninguna de estas propiedades la tiene t-SNE.

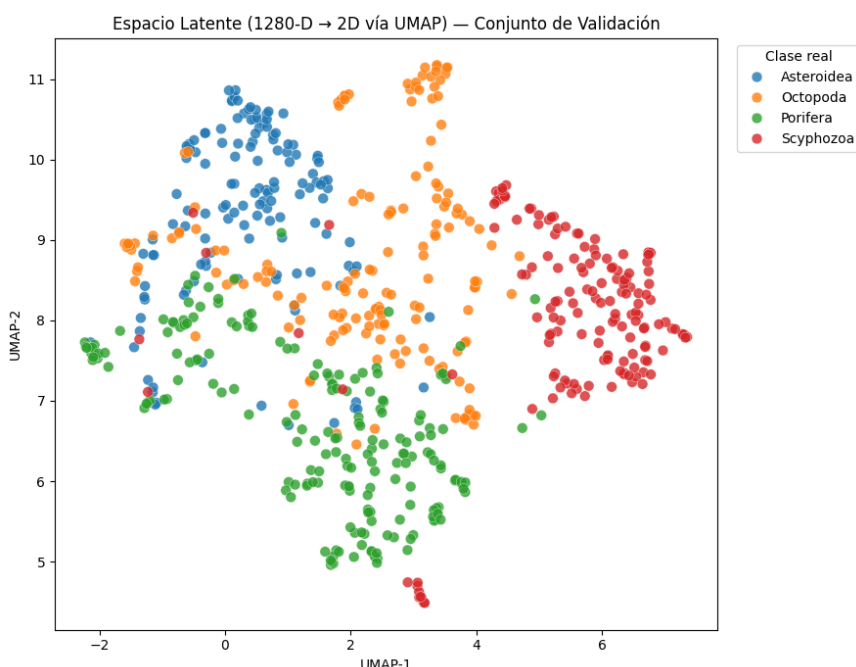


Fig. 1 — Proyección UMAP de los embeddings de validación (EfficientNet-B0, 1280-D). Scyphozoa se separa con claridad; Asteroidea, Octopoda y Porifera muestran solapamiento parcial en el espacio reducido.

3.2 Primer intento: Isolation Forest (Celdas 16-18)

Se eligió Isolation Forest como primer detector por ser *no-paramétrico y eficiente en alta dimensión, sin necesitar calcular distancias par-a-par*. Entrenado exclusivamente sobre embeddings de **train** (nunca val, para no contaminar el criterio de “normalidad” con datos de evaluación).

HALLAZGO: Bug de calibración: contamination="auto"

La primera corrida con contamination="auto" dio una falsa alarma de **0.00%** — ningún caso marcado como "Desconocido", en las 4 clases. Investigando la causa (Celda 17): scikit-learn fija con ese modo un umbral de decisión constante (offset_ = -0.5) que **no se calibra contra la escala real de los datos**. La distribución empírica de decision_function en este embedding vivía en el rango [0.01, 0.14] — completamente alejada de -0.5. El detector no era preciso: estaba roto. Se recalibró usando el percentil 2% de la distribución de *train* (contamination=0.02), **obteniendo una falsa alarma de 1.23% en validación**.

3.3 La prueba de la verdad: validación contra una especie real nunca vista (Celdas 19-21)

El conjunto de validación contiene únicamente las 4 clases conocidas — por lo tanto, la falsa alarma del 1.23% solo mide **especificidad** (no marcar lo conocido como desconocido), nunca **sensibilidad** real (detectar algo genuinamente nuevo). Para medir esto último se descargó de FathomNet un conjunto independiente de **Holothuroidea** (pepinos de mar, n=120) — taxón nunca usado en entrenamiento ni validación, morfológicamente distinto de las 4 clases conocidas.

Resultado: **3.33%** de detección sobre las Holothuroidea, frente al 1.23% de falsa alarma basal. Una prueba de dos proporciones dio **$z = 1.71$, $p = 0.087$** — **no estadísticamente significativo**. No se pudo demostrar que el detector distinguiera mejor que el ruido aleatorio entre una especie nueva y una conocida.

Diagnóstico adicional (Celda 21): se compararon las distribuciones de decision_function de train, val y OOD. Las tres curvas resultaron **prácticamente superpuestas** — no era un problema de calibración del umbral (que se podría arreglar bajando el percentil), sino de **representación**: el embedding, optimizado para discriminar solo 4 clases vistas en entrenamiento, no codifica una señal de "novedad" generalizable a categorías nunca observadas — una limitación documentada en la literatura de OSR sobre redes discriminativas de conjunto cerrado.

3.4 Pivote: Distancia de Mahalanobis a Centroides de Clase (Celdas 22-27)

Se decidió no descartar el embedding, sino probar un detector de naturaleza distinta: distancia de **Mahalanobis** al centroide de clase más cercano (Lee et al., 2018). A diferencia de Isolation Forest (particiones axis-aligned), Mahalanobis modela densidad gaussiana por clase considerando la correlación entre las 1280 dimensiones — un mecanismo completamente distinto que, de fallar también, reforzaría la hipótesis de que el problema es del embedding y no del algoritmo.

La covarianza se estimó con **contracción de Ledoit-Wolf**, necesaria porque con 1280 dimensiones y solo ~300 muestras por clase la covarianza empírica es singular (más dimensiones que datos) y no se puede invertir.

HALLAZGO: Segundo bug de calibración: centroides auto-referenciados

La primera calibración (percentil 98% de la distancia de *train* a sus propios centroides) dio una falsa alarma **inflada** de 27.65% en validación, frente al ~2% esperado. Causa: un centroide es, por construcción, el punto que minimiza la distancia a los datos que lo definieron — la distancia de un punto de train a su propio centroide es sistemáticamente menor que la de cualquier punto nuevo de la misma distribución, incluso sin overfitting real (el mismo motivo estructural por el que el error de entrenamiento es menor al de test). **Corrección**: se calibró el umbral con una mitad de validación (val_calib, que nunca participó del cálculo de centroides) y se midió la falsa alarma sobre la otra mitad (val_test) — el mismo principio anti-leakage aplicado en toda la fase. Resultado corregido: **2.15%** de falsa alarma, **14.17%** de detección OOD, con **$z = 4.99$, $p < 0.001$** — ahora sí estadísticamente significativo.

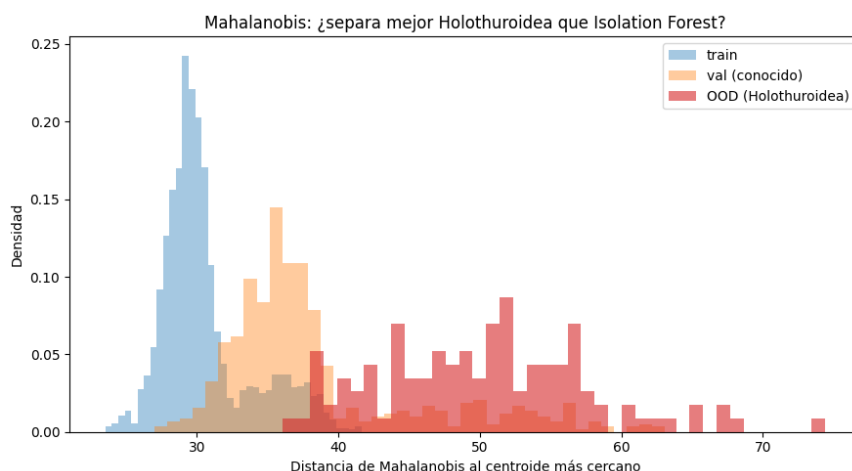


Fig. 2 — Distribución de distancias de Mahalanobis: train, validación (clases conocidas) y OOD real (Holothuroidea). A diferencia de Isolation Forest, la curva OOD se separa claramente hacia la derecha de train/val.

Calibración final: barrido de percentiles y decisión por costo asimétrico

Se barrieron percentiles de calibración (80% a 99%) midiendo en cada caso falsa alarma y detección OOD. La elección final no se basó en maximizar una métrica aislada, sino en el **costo asimétrico de los errores** en el contexto de uso real: el sistema es un filtro de apoyo para revisión humana, no un clasificador autónomo. Un falso positivo cuesta unos segundos de revisión manual de una imagen ya conocida; un falso negativo significa **perder la oportunidad de detectar una especie genuinamente nueva** durante la expedición — un costo mucho mayor. Se priorizó por lo tanto **sensibilidad** sobre especificidad, eligiendo el percentil 90%.

Percentil calibración	Falsa alarma (val)	Detección OOD	z	p-value
80%	13.19%	53.33%	8.80	< 0.001
95%	5.52%	28.33%	6.66	< 0.001
98%	2.15%	14.17%	4.99	< 0.001
99%	1.53%	10.00%	4.14	< 0.001

Sistema final adoptado (percentil 90%): **13.19%** falsa alarma en validación, **53.33%** de detección sobre Holothuroidea real.

Comparación final de detectores

Sistema	Falsa alarma (val)	Detección OOD real	z	¿Significativo ?
Isolation Forest	1.23%	3.33%	1.71	No (p=0.087)
Mahalanobis (umbral p90)	13.19%	53.33%	8.80	Sí (p<0.001)

Cierre visual: aciertos y omisiones reales

La inspección de las imágenes reales detectadas y omitidas reveló un patrón morfológico claro: las Holothuroidea correctamente detectadas presentan textura visible (espinas, papilas, segmentación), mientras que las omitidas tienen un cuerpo liso y de bajo detalle textural — generando un embedding más “genérico”, más cercano a las clases conocidas. Las distancias de los casos omitidos quedaron apenas por debajo del umbral, sugiriendo un margen ajustado más que una falla completa de representación.



Fig. 3 — Top: Holothuroidea detectadas correctamente como "Desconocido" (textura visible). Bottom: casos omitidos (cuerpo liso, distancia justo bajo el umbral).

También se identificó, por inspección de la matriz de Isolation Forest, que su mayor falsa alarma (Octopoda) se concentraba en recortes con **problemas de calidad de imagen** (baja resolución, pixelado) — un hallazgo secundario no buscado: el sistema de anomalías puede funcionar también como filtro de control de calidad de datos.

4. Fase 4: Comparación de Arquitecturas

Con el sistema de OSR cerrado sobre EfficientNet-B0, se evaluó si el backbone elegido en la Fase 2 era realmente la mejor opción disponible, comparándolo contra 5 arquitecturas adicionales: MobileNetV3-Small, ResNet18, ViT-Tiny, EfficientNetV2-S y ConvNeXt-Tiny.

4.1 Harness genérico de entrenamiento (Celda 28)

En vez de duplicar el código de entrenamiento de las Celdas 9-10-12 seis veces, se construyó un *harness* único parametrizado por arquitectura. Esto fue deliberado por una razón metodológica: entrenar las 6 arquitecturas bajo el **mismo protocolo** (mismos hiperparámetros, mismo dataset, mismo criterio de early stopping) aísla la arquitectura como **única variable bajo prueba** — control de variables experimental. Duplicar código, además, es una fuente directa de bugs por inconsistencia entre copias.

4.2 Sesgo de protocolo descubierto y corregido (Celdas 30-30b)

HALLAZGO: EfficientNet-B0 estaba en desventaja por receta, no por arquitectura

Al sumar el EfficientNet-B0 ya entrenado (Fase 2, $F1=0.910$) a la tabla comparativa, **las 5 arquitecturas nuevas, sin excepción, superaron a B0** por márgenes grandes y parejos — una señal de alarma: si la arquitectura realmente importara tanto, se esperaría más dispersión entre resultados, no un salto uniforme. La causa identificada: B0 se entrenó con una receta distinta (learning rates diferenciales por bloque) a la del harness (LR uniforme) — se estaba comparando “arquitectura + receta A” contra “arquitectura + receta B”, no arquitecturas puras. **Corrección:** se reentrenó B0 bajo el mismo harness (Celda 30b). Resultado: **$F1 = 0.947$** — significativamente más alto que el original (0.910), confirmando el sesgo, pero **todavía el más bajo de las 6** arquitecturas comparadas.

4.3 Resultados

Arquitectura	Parámetros (M)	F1 macro val	Inferencia (ms/img)
ConvNeXt-Tiny	27.82	0.9905	29.73
EfficientNetV2-S	20.18	0.9831	22.80
ViT-Tiny	5.53	0.9818	12.52
ResNet18	11.18	0.9775	5.49
MobileNetV3-Small	1.52	0.9612	15.80
EfficientNet-B0 (receta unificada)	4.01	0.9469	13.57

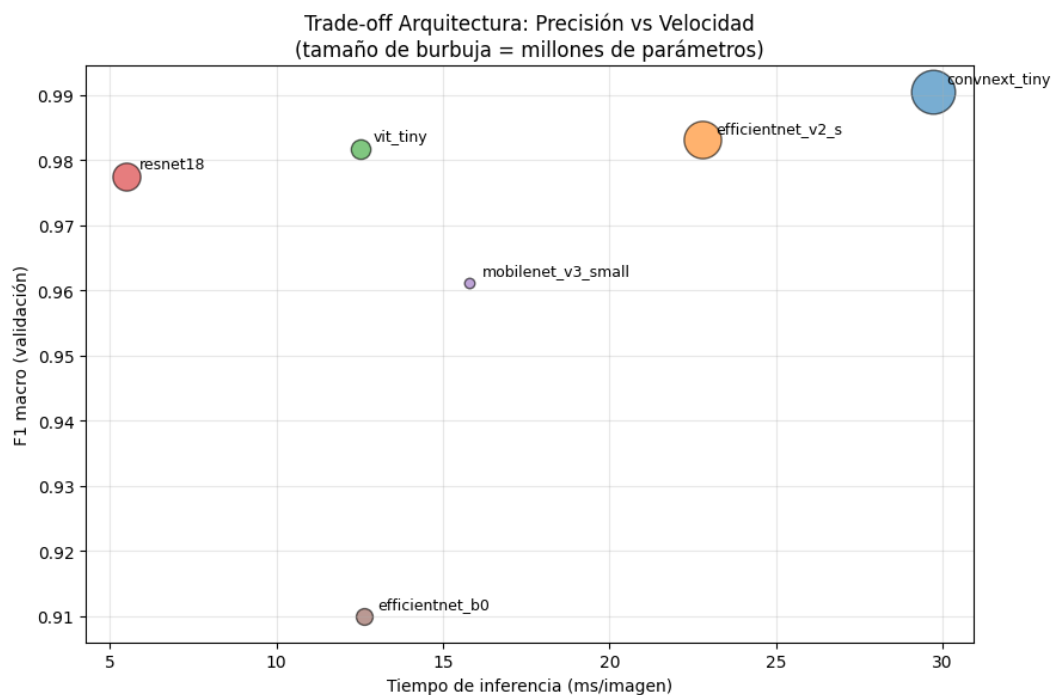


Fig. 4 — Trade-off precisión vs. velocidad de inferencia (tamaño de burbuja = parámetros). Gráfico generado tras excluir la receta original sesgada de B0.

4.4 Contexto que reformuló la decisión

Una primera lectura ingenua del gráfico llevaría a elegir ConvNeXt-Tiny (mayor F1). Pero el contexto real de uso (Sección 1.2) cambia el peso de cada criterio:

- **La velocidad casi no importa.** Con muestreo avanzado sobre video ya grabado (estimado en ~3000 frames por cada par de horas), la diferencia total entre la arquitectura más rápida (ResNet18) y la más lenta (ConvNeXt-Tiny) es de poco más de un minuto sobre un proceso batch — no es el criterio que debería decidir.
- **El embedding alimenta directamente la Fase 3 (OSR).** Cambiar de arquitectura implica rehacer todo el sistema de Mahalanobis. La dimensionalidad del nuevo embedding importa: con 1280-D, la covarianza de Ledoit-Wolf necesitó un *shrinkage* de 0.876 (87.6% de la matriz fue casi pura suposición, no señal real) — un régimen frágil. ResNet18 (512-D) ofrece una relación muestras/dimensión mucho más favorable para ese mismo cálculo.
- **No se conoce el hardware real de las computadoras del buque.** Sin GPU dedicada confirmada, conviene no apostar por la arquitectura más pesada del lote.
- **Riesgo de sobreajuste.** ConvNeXt-Tiny y EfficientNetV2-S son 5-7 veces más grandes que B0 sobre un dataset de ~2500 imágenes de entrenamiento — su F1 “ganador” es también el más sospechoso de estar inflado. ViT-Tiny, además, pertenece a una familia (Transformers de visión) documentada como particularmente *data-hungry*.

DECISIÓN FINAL: Arquitectura elegida: ResNet18

No por tener el F1 más alto (no lo tiene) ni por ser la más rápida (ese criterio pesa poco en este contexto). Se elige por mejorar **simultáneamente tres frentes** sin introducir una incertidumbre nueva: F1 sólido (+3 puntos sobre B0, con bajo riesgo de sobreajuste dada su madurez y tamaño moderado), un embedding de 512-D mucho más favorable para el sistema de OSR ya validado, y la opción más robusta si el hardware del buque resulta ser modesto. Es una decisión de **menor riesgo**, no de métrica máxima — y se documenta explícitamente así, incluyendo la advertencia de que se trata de una sola corrida por arquitectura, sin repetición de semillas.

5. Fase 5: Recalibración del Sistema de OSR con ResNet18

Cambiar el backbone invalida todo el sistema construido sobre los embeddings de EfficientNet-B0 (Celdas 14-27): los vectores de una red distinta no son comparables ni intercambiables con los anteriores. Esta fase repite el procedimiento de calibración, pero con dos decisiones de eficiencia tomadas a partir de lo ya aprendido:

- **No se repite el paso de Isolation Forest.** Ya se demostró con evidencia que no generaliza a OOD real; el problema es el objetivo de entrenamiento discriminativo cerrado, no una particularidad de B0 — no hay motivo para esperar un resultado distinto.
- **No se vuelve a descargar Holothuroidea de FathomNet.** Las imágenes ya están en disco desde la Fase 3 (Celda 19); solo se re-extraen sus embeddings con el nuevo extractor.

5.1 Procedimiento (Celdas 34-39)

Se decapitó ResNet18 (reemplazando .fc por Identity, a diferencia de .classifier en EfficientNet) para extraer embeddings de **512-D**. Se calculó la covarianza de Ledoit-Wolf y se calibró el umbral de Mahalanobis **directamente con la versión correcta** (split val_calib/val_test) desde el inicio, sin repetir el descubrimiento del sesgo de centroides auto-referenciados ya documentado en la Fase 3.

Se aplicó el mismo criterio de calibración ya acordado (percentil 90%, priorizando sensibilidad por el costo asimétrico de los errores) y se evaluó la sensibilidad del sistema frente al mismo conjunto OOD real (Holothuroidea), permitiendo una comparación directa, celda por celda, contra el sistema construido sobre EfficientNet-B0.

6. Optimización de Costos de GPU: Idempotencia en el Entrenamiento

El cupo gratuito de GPU de Colab es limitado, y el proyecto llegó a agotarlo en una sesión. Se identificó que, aunque los checkpoints (.pth) de cada modelo ya se guardaban en Drive, **ninguna celda verificaba si ya existían antes de entrenar** — cada vez que se volvía a correr una celda de entrenamiento (por reinicio de sesión, o simplemente para revisar el notebook completo), se repetía el entrenamiento completo desde cero, desperdiciando minutos u horas de GPU ya “pagados”.

SOLUCIÓN: Mismo principio que la extracción de datos, aplicado al entrenamiento

La Celda 3 (extracción FathomNet) ya implementaba “idempotencia robusta”: si una imagen ya estaba descargada, no se volvía a pedir. Se extendió el mismo principio a las celdas de entrenamiento (Celda 10, Celda 12, y la función `benchmark_arquitectura` de la Celda 28): antes de entrenar, cada celda verifica si ya existe el checkpoint **y** un archivo JSON con el historial/resultado de una corrida anterior. Si ambos existen, se cargan directamente — **0 minutos de GPU gastados** — en vez de reentrenar. Un parámetro explícito (`forzar=True` o `FORZAR_REENTRENAMIENTO = True`) permite invalidar el caché a propósito si se cambian hiperparámetros.

Validación: se probó la lógica de caché ejecutando el código real de las tres celdas parcheadas contra un modelo y dataset sintéticos en dos pasadas consecutivas — la primera entrena y guarda; la segunda detecta el caché y reusa el resultado sin tocar el modelo. Las dos pasadas produjeron historiales idénticos.

7. Una Decisión de Alcance: el Módulo de Lógica Difusa (RA1)

La propuesta de proyecto aprobada por la cátedra describe, además del clasificador CNN (RA2), un módulo de **lógica difusa** que ajustaría la confianza de cada detección según metadatos oceanográficos (profundidad, temperatura) — pensado para cubrir el resultado de aprendizaje RA1 (Conocimiento e Incerteza). Al revisar el avance del proyecto, se identificó que ese módulo nunca se había implementado.

Se diseñó una propuesta concreta y viable: funciones de membresía derivadas **empíricamente** de la distribución de profundidad/temperatura observada por clase en el propio dataset de entrenamiento (en vez de literatura biológica externa, inapropiada para taxones tan amplios como “Porifera” o “Asteroidea”, que abarcan rangos batimétricos enormes), normalizando a z-score para evitar diseñar 8 conjuntos difusos distintos, y con manejo explícito de metadatos faltantes (~38-39% de las imágenes no tienen profundidad/temperatura registrada) mediante abstención — el sistema no penaliza por falta de dato.

DECISIÓN: Se descartó implementarlo

Pese a estar contemplado en la propuesta y a contar con un diseño viable, se decidió **no implementar el módulo difuso**: el tema no fue estudiado ni practicado en la cursada (apenas se nombró). Implementar a último momento una técnica no comprendida en profundidad es más riesgoso para la defensa que no implementarla — el equipo no podría defenderla con solidez ante preguntas del tribunal, y el rubro de evaluación valora explícitamente la **honestidad intelectual** al reportar limitaciones. Se priorizó profundizar en las técnicas efectivamente trabajadas en clase (redes neuronales, detección de anomalías) antes que sumar superficialmente una técnica nueva.

8. Limitaciones Conocidas y Trabajo Futuro

Documentar limitaciones con la misma evidencia con la que se documentan los aciertos es parte del estándar de rigurosidad que guió todo el proyecto. Se listan las más relevantes:

- **Sensibilidad OOD acotada.** Incluso el mejor sistema (Mahalanobis, percentil 90%) detecta poco más de la mitad de las muestras de una especie genuinamente nueva (53.33%). Un sistema de producción debería complementarse con revisión humana, no operar de forma completamente autónoma.
- **Una sola corrida por arquitectura, sin repetición de semillas (Fase 4).** Las diferencias de 1-3 puntos de F1 entre ResNet18, ViT-Tiny y EfficientNetV2-S podrían no ser estadísticamente robustas frente a la variabilidad de inicialización.
- **Conjunto OOD de validación limitado a una sola especie (Holothuroidea, n=120).** No se puede generalizar la tasa de detección a cualquier organismo desconocido; distintas morfologías podrían comportarse de forma distinta frente al detector (como ya se observó: Holothuroidea con textura se detecta mejor que Holothuroidea de cuerpo liso).
- **Calidad de imagen y heterogeneidad morfológica afectan la falsa alarma de forma desigual entre clases.** Octopoda concentra errores de Isolation Forest por calidad de recorte; Porifera concentra errores de Mahalanobis por diversidad morfológica intra-clase.
- **Hardware de despliegue real desconocido.** Las mediciones de tiempo de inferencia se hicieron sobre GPU T4 (Colab); el comportamiento relativo entre arquitecturas podría cambiar en CPU, que es plausiblemente el hardware real disponible en el buque.
- **Módulo de lógica difusa (RA1) no implementado,** por decisión deliberada de alcance (Sección 7), pese a estar diseñado.

Líneas de trabajo futuro:

- Ampliar el conjunto OOD de validación a múltiples taxones nunca vistos, para una estimación más robusta de la sensibilidad real del sistema.
- Repetir el entrenamiento de cada arquitectura de la Fase 4 con múltiples semillas, para cuantificar la varianza de las diferencias observadas.
- Explorar técnicas de *outlier exposure* o representaciones entrenadas explícitamente para detección de novedad, en vez de reusar embeddings de un clasificador discriminativo cerrado.
- Confirmar el hardware real de despliegue en el buque y re-medir tiempos de inferencia en esas condiciones.

9. Conclusión

El proyecto avanzó mediante un patrón consistente: **implementar, validar con evidencia (no solo con la métrica más favorable), y corregir cuando la evidencia contradice la expectativa inicial**. Ese patrón se repitió en los tres pivotes documentados en esta bitácora:

- Isolation Forest parecía funcionar (1.23% de falsa alarma) hasta que una prueba contra una especie real nunca vista reveló que no detectaba nada mejor que el azar — el número bueno escondía un detector roto.
- La primera calibración de Mahalanobis parecía funcionar incluso mejor (95.83% de detección) hasta que se identificó que el umbral estaba sesgado por construcción — el número extraordinario escondía una metodología incorrecta.
- EfficientNet-B0 parecía claramente inferior a las otras 5 arquitecturas (0.910 vs. 0.96-0.99 de F1) hasta que se identificó que la comparación no controlaba el protocolo de entrenamiento — la brecha grande escondía, en parte, un sesgo experimental.

En los tres casos, la respuesta no fue descartar el resultado sospechoso ni aceptarlo sin más: fue investigar la causa, corregirla, y volver a medir. El sistema final — clasificador ResNet18 fine-tuneado, con detección de anomalías por distancia de Mahalanobis calibrada sin sesgo y validada contra una especie real nunca vista — es la versión de esa cadena de decisiones que mejor resiste la pregunta “¿por qué elegiste esto y no otra cosa?”, con evidencia concreta detrás de cada respuesta.

Documento generado como respaldo técnico del Trabajo Final Integrador — Inteligencia Computacional (IC415). Todos los números, decisiones y hallazgos citados corresponden a ejecuciones reales documentadas en el notebook del proyecto.